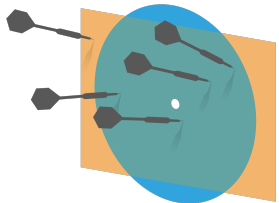


Probability and Statistics for Computer Science

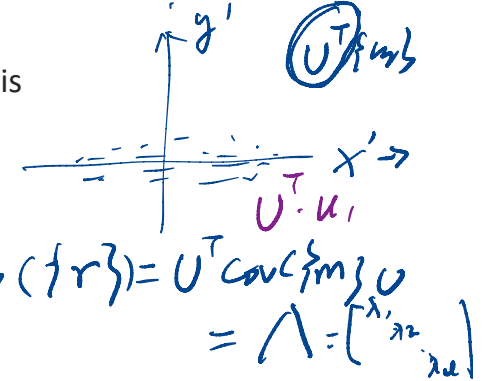
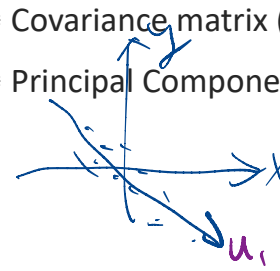


Credit: wikipedia

"...many problems are naturally classification problems"---Prof. Forsyth

Last time

- * Covariance matrix (II)
- * Principal Component Analysis



Objectives

- * Introduction to classification
- * Nearest Neighbor Classifier
- * Decision tree
- * Random forest

Introduction to classification

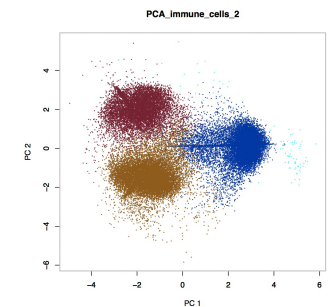
- * Given a set of feature vectors x_i , where each has a class label y_i , we want to train a classifier that maps unlabeled data with the same features to its label.

feature vectors

CD45	CD19	CD11b	CD3e	Type
6.59564671	1.297765164	7.073280884	1.155202366	1
6.742586812	4.692018952	3.145976639	1.572686963	4
6.300680301	1.20613983	6.393630905	1.424572629	2
5.455310882	0.958837541	6.149306002	1.493503124	1
5.725565772	1.719787885	5.998232014	1.310208305	1
5.552847151	0.881373587	6.02155471	0.881373587	3

5 0.5 4 2 ?

label



Binary classifiers

- ⌘ A binary classifier maps each feature vector to one of two classes.
- ⌘ For example, you can train the classifier to:
 - ⌘ Predict a gain or loss of an investment
 - ⌘ Predict if a gene is beneficial to survival or not
 - ⌘ ...

*student
or staff?*

Multiclass classifiers

- ⌘ A multiclass classifier maps each feature vector to one of three or more classes.
- ⌘ For example, you can train the classifier to:
 - ⌘ Predict the cell type given cells' measurement
 - ⌘ Predict if an image is showing tree, or flower or car, etc
 - ⌘ ...

How do we know if our classifier is good?

- ⌘ We want the classifier to avoid some mistakes on unlabeled data that we will see in run time.
- ⌘ **Problem 1:** some mistakes may be more costly than others
We can tabulate the types of error and define a loss function
- ⌘ **Problem 2:** It's hard to know the true labels of the run-time data
We must separate the labeled data into a training set and test/validation set

Performance of a binary classifier

- ⌘ A binary classifier can make two types of errors
 - ⌘ False positive (FP)
 - ⌘ False negative (FN)
- ⌘ Sometimes one type of error is more costly
 - ⌘ Drug effect test
 - ⌘ Crime detection
- ⌘ We can tabulate the performance in a class confusion matrix

Actual	Predicted	
	Negative	Positive
	Negative	Positive
Negative	True Negative	False Positive
Positive	False Negative	True Positive

TN	FP
15	3
FN	TP
7	25

Performance of a binary classifier

- A loss function assigns costs to mistakes
- The 0-1 loss function treats FPs and FNs the same
 - Assigns loss 1 to every mistake
 - Assigns loss 0 to every correct decision
- Under the 0-1 loss function
 - accuracy = $\frac{TP + TN}{TP + TN + FP + FN}$
 - The baseline is 50% which we get by random decision.

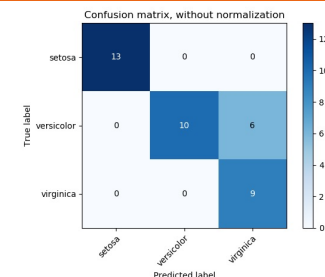
Actual	Predicted	
	Negative	Positive
	Negative	Positive
	True Negative	False Positive
	False Negative	True Positive

Performance of a multiclass classifier

- Assuming there are c classes:
- The class confusion matrix is $c \times c$
- Under the 0-1 loss function

$$\text{accuracy} = \frac{\text{sum of diagonal terms}}{\text{sum of all terms}}$$

ie. in the right example, accuracy = $32/38 = 84\%$
- The baseline accuracy is $1/c$.



Training set vs. validation/test set

- We expect a classifier to perform worse on run-time data
 - Sometimes it will perform much worse: an **overfitting** in training
 - An extreme case is: the classifier correctly labeled 100% when the input is in the training set, but otherwise makes a random guess
- To protect against overfitting, we separate training set from validation/test set
 - Training set** for training the classifier
 - Validation/test set** is for evaluating the performance
- It's common to reserve at least 10% of the data for testing

Cross validation with training and testing

- Cross-validation** is randomly splitting the labeled data to training and testing to see if our training method is sound.
- Common methods:
 - Leave one out (for smaller data set)
 - Dividing the size of data by k (fold) and use one of the fold for testing.
- Compare the accuracies built from different models

How many trained models I can have for the leave one out cross-validation?

If I have a data set that has 50 labeled data entries, how many leave-one-out validations I can have?

- A. 50
- B. 49
- C. 50×49

$$\binom{50}{1}$$

How many trained models can I have with this cross-validation?

If I have a data set that has 51 labeled data entries, I divide them into three folds. How many trained models can I have?

$$\binom{51}{17} > 1.47 \times 10^{13}$$

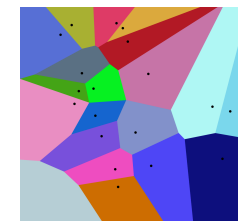
*The common practice of using fold is to divide the samples into equal sized k groups and reserve one of the group as the test data set.

Review questions about classifiers

- * Why do we need classifiers?
- * What do we use to quantify the performance of a classifier?
- * What is the baseline accuracy of a 5-class classifier using 0-1 loss function?
- * What's validation and cross-validation in classification?

Nearest neighbor's classifier (KNN)

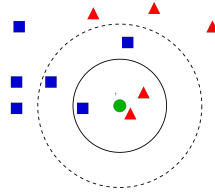
- * Given an unlabeled feature vector
 - * Calculate the distance from x
 - * Find the closest labeled x_i
 - * Assign the same label to x
- * Practical issues
 - * We need a distance metric
 - * We should first standardize the data
 - * Classification may be less effective for very high dimensions



Source: wikipedia

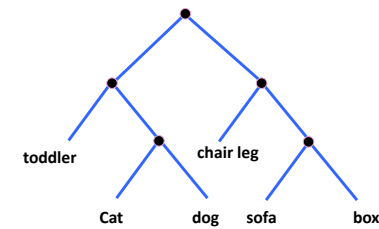
Variants of nearest neighbor's classifier

- In k-nearest neighbors, the classifier:
 - Looks at the k nearest labeled feature vectors $\mathbf{x}_i$
 - Assigns a label to $\mathbf{x}$ based on a majority vote
- In (k, ℓ)-nearest neighbors, the classifier:
 - Looks at the k nearest labeled feature vectors
 - Assigns a label to $\mathbf{x}$ if at least ℓ of them agree on the classification



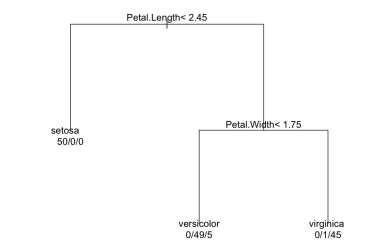
Decision tree: object classification

- The object classification **decision tree** can classify objects into multiple classes using sequence of simple tests. It will naturally grow into a tree.

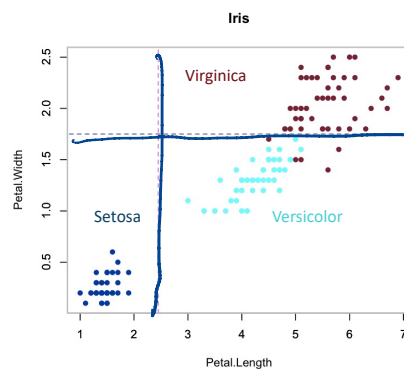


Training a decision tree: example

- The "Iris" data set



Features: Sepal.Length, Sepal.Width, Petal.Length, Petal.Width
Label: Species

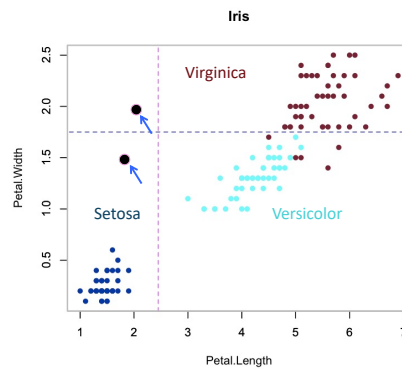
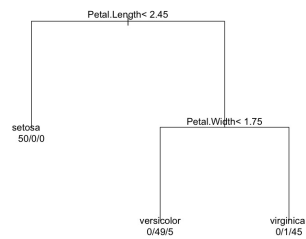


Training a decision tree

- Choose a dimension/feature and a split
- Split the training Data into left- and right- child subsets D_L and D_R
- Repeat the two steps above recursively on each child
- Stop the recursion based on some conditions
- Label the leaves with class labels

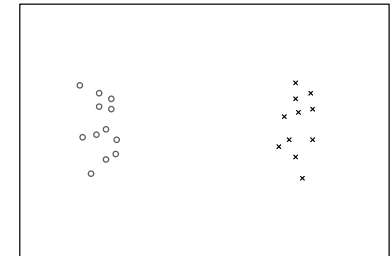
Classifying with a decision tree: example

The "Iris" data set



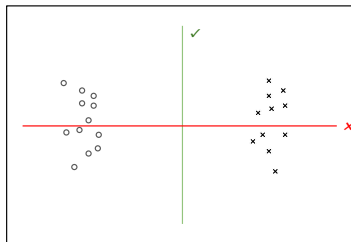
Choosing a split

- An informative split makes the subsets more concentrated and reduces uncertainty about class labels



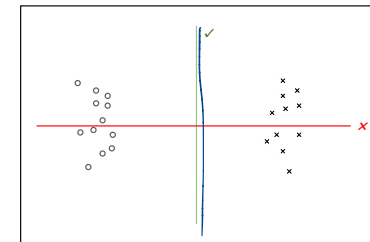
Choosing a split

- An informative split makes the subsets more concentrated and reduces uncertainty about class labels

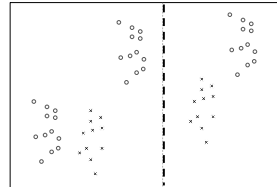
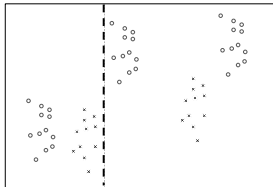


Choosing a split

- An informative split makes the subsets more concentrated and reduces uncertainty about class labels

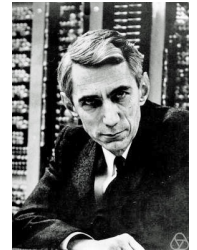


Which is more informative?



Quantifying uncertainty using entropy

- ⌘ We can measure uncertainty as the number of bits of information needed to distinguish between classes in a dataset (first introduced by Claude Shannon)
- ⌘ We need $\log_2 2 = 1$ bit to distinguish 2 equal classes
- ⌘ We need $\log_2 4 = 2$ bits to distinguish 4 equal classes



Claude Shannon (1916-2001)

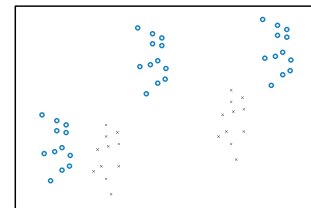
Quantifying uncertainty using entropy

- ⌘ **Entropy** (Shannon entropy) is the measure of uncertainty for a general distribution
- ⌘ If class i contains a fraction $P(i)$ of the data, we need $\log_2 \frac{1}{P(i)}$ bits for that class
- ⌘ The entropy $H(D)$ of a dataset is defined as the **weighted mean** of entropy for every class:

$$H(D) = \sum_{i=1}^c P(i) \log_2 \frac{1}{P(i)}$$

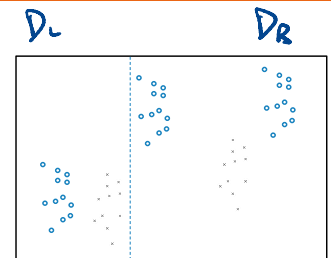
$$= \sum_{i=1}^c -p \log_2 p$$

Entropy: before the split



$$H(D) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5}$$

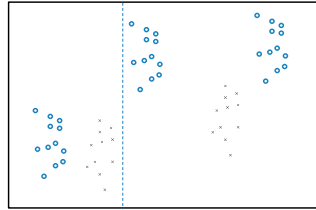
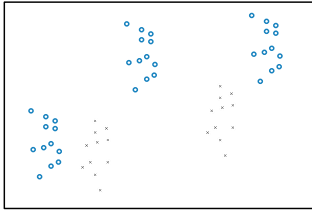
$$= 0.971 \text{ bits}$$



$$H(D_L) = -\sum p \log_2 p = -\frac{1}{5} \log_2 \frac{1}{5} - \frac{4}{5} \log_2 \frac{4}{5}$$

$$H(D_R) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 1$$

Entropy example: after the split



$$H(D) = -\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5}$$

$$= 0.971 \text{ bits}$$

Information gain of a split

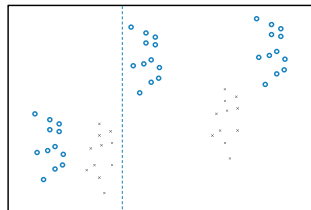
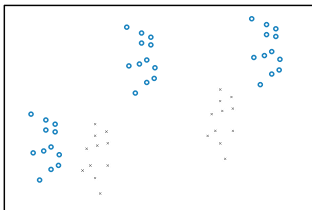
- The information gain of a split is the amount of entropy that was reduced on average after the split

$$I = H(D) - \left(\frac{N_{D_l}}{N_D} H(D_l) + \frac{N_{D_r}}{N_D} H(D_r) \right)$$

- where

- N_D is the number of items in the dataset D
- N_{D_l} is the number of items in the left-child dataset D_l
- N_{D_r} is the number of items in the right-child dataset D_r

Information gain for the previous example



$$I = H(D) - \left(\frac{N_{D_l}}{N_D} H(D_l) + \frac{N_{D_r}}{N_D} H(D_r) \right)$$

$$= 0.971 - \left(\frac{2}{5} \times 1 + \frac{3}{5} \times 0.918 \right) = 0.0202$$

Q. Is the splitting method global optimum?

- A. Yes
- ☒ B. No

How to choose a dimension and split

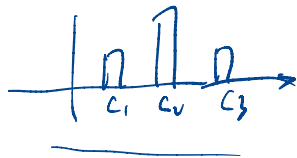
- ✱ If there are d dimensions, choose approximately $\sqrt{d}$ of them as candidates at random
- ✱ For each candidate, find the split that maximizes the information gain
- ✱ Choose the best overall dimension and split
- ✱ Note that splitting can be generalized to categorical features for which there is no natural ordering of the data

When to stop growing the decision tree?

- ✱ Growing the tree too deep can lead to overfitting to the training data
- ✱ Stop recursion on a data subset if any of the following occurs:
 - ✱ All items in the data subset are in the same class
 - ✱ The data subset becomes smaller than a predetermined size
 - ✱ A predetermined maximum tree depth has been reached.

How to label the leaves of a decision tree

- ✱ A leaf will usually have a data subset containing many class labels
- ✱ Choose the class that has the most items in the subset
- ✱ Alternatively, label the leaf with the number it contains in each class for a probabilistic “soft” classification.



Pros and Cons of a decision tree

- ✱ Pros: Flexible
OK for speed
Interpretable
- ✱ Cons: Overfitting

Assignments

- ✿ Read Chapter 11 of the textbook
- ✿ Next time: Random forest, SVM, Stochastic gradient descent
- ✿ Examlet3 at CBTF

Additional References

- ✿ Charles M. Grinstead and J. Laurie Snell "Introduction to Probability"
- ✿ Morris H. Degroot and Mark J. Schervish "Probability and Statistics"
- ✿ Mor Harchol-Balter "Introduction to Probability for Computing"

See you next time

See you!



